

INTERVIEW PREPARATION

Questions & Answers

Everything an interviewer can ask about the NovaBank project

158 questions across 12 topics

Answers written the way you would say them out loud - correct, specific, and grounded in the code you actually wrote.

● The Project	12	● Java and OOP	14
● Spring Boot	16	● JPA and Hibernate	16
● Database and SQL	12	● Security and JWT	16
● Concurrency and Transactions	16	● REST API Design	10
● React and Frontend	18	● Testing	10
● Docker and Deployment	10	● Behavioural and Closing	8

Gurudutta Pradhan

guruduttapradhan140@gmail.com

Contents

1	The Project	Opening questions - the ones that are asked first, every time	Q1-Q12
2	Java and OOP	Language fundamentals, with examples from this codebase	Q13-Q26
3	Spring Boot	The framework, its magic, and where the magic stops	Q27-Q42
4	JPA and Hibernate	Persistence, mapping, and the traps	Q43-Q58
5	Database and SQL	Schema design, indexing, and query behaviour	Q59-Q70
6	Security and JWT	Authentication, authorization, and the threat model	Q71-Q86
7	Concurrency and Transactions	The heart of the system - expect the most questions here	Q87-Q102
8	REST API Design	Contracts, status codes, and versioning	Q103-Q112
9	React and Frontend	Components, state, performance, and design	Q113-Q130
10	Testing	What is tested, how, and why it is convincing	Q131-Q140
11	Docker and Deployment	Containers, configuration, and operations	Q141-Q150
12	Behavioural and Closing	Questions about you, answered through the project	Q151-Q158

How to use this document

Do not memorise these answers - an interviewer can tell. Read them until you understand the mechanism, then say it in your own words. The sections on Concurrency and Transactions and on Security are where a strong interviewer will spend their time, because that is where this project is genuinely interesting. If you can only prepare two sections, prepare those.

The three answers to have ready cold

1. The lost-update problem and how row locking with a consistent lock order solves it. 2. Why the retry had to move outside the transaction boundary, and what `@Transactional`'s proxy has to do with it. 3. Why there are two tokens instead of one, and what refresh rotation buys you. Almost every deep technical question in this document routes back to one of those three.

CHAPTER 1

The Project

Opening questions - the ones that are asked first, every time

1

Q1 Tell me about this project in two minutes.

NovaBank is a full-stack bank management system: a React 19 single-page app talking to a Spring Boot 3.5 REST API backed by PostgreSQL. Customers register, open Savings, Current or Fixed Deposit accounts, deposit and withdraw, transfer money, save payees, filter their history and download CSV statements. Administrators get a separate back office with bank-wide metrics, account freezing and an audit trail.

What makes it worth talking about is not the CRUD - it is the money movement. A transfer is atomic across two accounts, safe under concurrent access through pessimistic row locking with a consistent lock order, idempotent through a client-supplied key backed by a unique index, and recorded as a double-entry pair so the ledger can always be reconciled. There are 27 tests, including one that fires 20 simultaneous transfers and asserts that not a single rupee was created or destroyed.

Q2 Why did you choose banking as the domain?

Because it forces you to solve problems a CRUD app never raises. A to-do list does not care if two requests arrive at once. A bank does: two withdrawals hitting the same balance simultaneously can both be approved and the bank silently creates money. A transfer touches two rows and must be all-or-nothing. A dropped network connection means a client will retry, and a retried transfer must not charge twice. Floating-point arithmetic quietly loses fractions of currency.

Every one of those has a specific engineering answer in the codebase, and each one is a real conversation rather than a feature list.

Q3 What was the hardest problem you solved?

Concurrent transfers returning HTTP 500s. I fired twelve simultaneous transfers at the running API and only three succeeded - the other nine returned 500. The important part is what was *not* wrong: no money was lost or created, so the locking itself was correct.

The cause was that under contention a database refuses a transaction rather than queuing it - a lock-wait timeout, or being chosen as a deadlock victim, which surfaces as SQLSTATE 40001. Those are transient: nothing was applied, so the right response is to try again.

The subtle part was **where** the retry could live. Once a transaction is marked rollback-only, every further statement on it fails, so a retry loop inside the service method would just fail four more times inside the same doomed transaction. The retry has to start a brand-new transaction, which means it must sit outside the transactional boundary - and because Spring's `@Transactional` works through a proxy, that means a different bean. I extracted `MoneyMovementFacade` with a bounded retry and jittered backoff. After the fix, 12 of 12 applied, and lock failures that survive all retries return a retryable 409 instead of a 500.

Q4 What would you do differently if you started again?

Three things. **TypeScript on the frontend** from day one, with types generated from the OpenAPI document, so the client and server literally cannot disagree about a payload shape. **Testcontainers instead of H2** for integration tests, so the tests run against the same PostgreSQL version as production - H2's locking semantics differ, and that difference is exactly what my concurrency tests are trying to verify. And **httpOnly cookies instead of localStorage** for tokens, accepting the CSRF work that comes back with it.

Q5 What are the weaknesses of this system?

Tokens in localStorage are readable by any JavaScript on the page, so a successful XSS steals the session. There is no rate limiting, so login is brute-forceable given time. It is single-currency with no FX. Money only moves between NovaBank accounts - there are no external rails. Interest is a simple monthly accrual rather than daily compounding on minimum balance. KYC is a boolean an admin toggles, not a workflow. And there is no two-factor authentication.

I would rather state these than have an interviewer find them. Knowing the boundaries of your own system is a stronger signal than claiming it has none.

Q6 How long did it take and how did you approach it?

I designed the data model and the transfer algorithm first, because everything else depends on them - getting the ledger shape wrong would have meant rewriting the services. Then the backend from the inside out: entities, repositories, services, controllers, with tests alongside the business rules. Then the API contract stabilised, which let the frontend be built against something that would not shift under it. Docker and the documentation last.

Q7 Is this production-ready?

No, and I would not claim it is. The core is correct - money is conserved, concurrency is safe, actions are audited - but a production deployment needs TLS termination, secrets from a manager rather than environment variables, rate limiting, centralised structured logging with correlation ids, metrics and alerting, database backups with a tested restore, and a CI pipeline with dependency scanning. Chapter 12 of the documentation lists them explicitly.

Q8 Which part of the code are you proudest of?

The lock-ordering in `TransactionService.transfer()`. It is four lines, it is the difference between a system that works and one that deadlocks in production, and it is invisible unless you know why it is there. Sorting the two accounts by number and always locking the lower one first makes a circular wait structurally impossible rather than merely unlikely.

```
String first = min(request.fromAccountNumber(), request.toAccountNumber());
String second = max(request.fromAccountNumber(), request.toAccountNumber());
Account firstLocked = lockAccount(first);
Account secondLocked = lockAccount(second);
```

Q9 How would you scale this to a million users?

In order of what I would actually hit first. **Read replicas** - the read path is already separated into `TransactionQueryService` with `readOnly` transactions, so routing reads to a replica is a configuration change. **Connection pooling and query tuning** - the indexes are already aligned to the hot queries. **Horizontal scaling of the API** - it is stateless, with no `HttpSession`, so instances scale out behind a load balancer with no sticky sessions.

Beyond that: caching the dashboard aggregates, partitioning the transactions table by date since it only grows, and moving audit writes onto an async queue. Eventually, sharding by customer - but that breaks cross-shard transfers and needs a saga or two-phase commit, so it is a last resort.

Q10 Why is the money-movement retry a separate class instead of a method?

Because Spring's `@Transactional` is implemented with a proxy. A call from one method to another **in the same class** goes through 'this' and never touches the proxy, so it joins the existing transaction rather than starting a new one. A retry that reuses the rollback-only transaction is useless. Crossing a bean boundary is what makes the proxy - and therefore the new transaction - real.

Q11 Walk me through what happens when a user clicks 'Send money'.

The React form validates locally for fast feedback, disables the submit button so a double-click cannot fire twice, and POSTs to `/api/v1/transactions/transfer` with an idempotency key minted for this form session. Axios attaches the bearer token.

Server side: `JwtAuthenticationFilter` verifies the signature and populates the `SecurityContext`; the filter chain confirms the route needs authentication; `@Valid` checks the body. `MoneyMovementFacade` enters its retry loop and calls `TransactionService.transfer()`, where the transaction begins. That method rejects a self-transfer, checks the idempotency key, locks both accounts in ascending account-number order, verifies ownership, checks status, spendable balance and daily limit, applies the debit and credit in memory, writes two ledger rows sharing one reference, and writes an audit entry on its own transaction. Commit. 201 with a receipt.

Q12 How do you know the system is actually correct?

By testing the invariant rather than the implementation. `TransferConcurrencyIT` starts 20 threads, holds them at a `CountDownLatch`, releases them simultaneously, and then asserts that the sum of both balances equals the sum they started with, that the sender's balance dropped by exactly the total moved, and that exactly 40 ledger rows exist for 20 transfers. A sequential test could never catch a lost update; this one would fail loudly.

CHAPTER 2

Java and OOP

Language fundamentals, with examples from this codebase

2

Q13 Why Java 21 specifically?

It is an LTS release, so it has long-term support rather than six months. Three features earn their place here: **records**, which make every DTO a single immutable line with correct equals, hashCode and toString; **pattern matching for switch and instanceof**, used throughout the exception handling; and **virtual threads**, enabled in application.yml, which let the server handle many concurrent blocking requests without a large platform-thread pool.

Q14 What is a record and why did you use records for DTOs?

A record is an immutable data carrier: you declare the components and the compiler generates the constructor, accessors, equals, hashCode and toString. For DTOs that is exactly right - a DTO has no behaviour and should never be mutated after construction. It turns a 60-line class into one line.

Records cannot be JPA entities, though - JPA needs a no-arg constructor and mutable fields for the proxying and dirty checking it does - so entities here are ordinary classes.

```
public record TransferRequest(
    @NotBlank String fromAccountNumber,
    @NotBlank String toAccountNumber,
    @NotNull @DecimalMin("0.01") BigDecimal amount,
    String description, String category, String idempotencyKey) {}
```

Q15 Why BigDecimal instead of double for money?

Because binary floating point cannot represent most decimal fractions exactly. The nearest double to 0.1 is slightly more than a tenth, so $0.1 + 0.2$ evaluates to 0.30000000000000004, and adding 0.1 ten times gives 0.9999999999999999. In a ledger, that drift is money that vanished, and it accumulates.

BigDecimal stores an arbitrary-precision unscaled value plus a scale, so decimal arithmetic is exact. Two rules go with it: always construct from a **String**, never from a double, because `new BigDecimal(0.1)` captures the floating-point error; and always compare with `compareTo`, because `equals()` also compares scale, so 100 and 100.00 are not equal.

```
jshell> 0.1 + 0.2
$1 ==> 0.30000000000000004

new BigDecimal("0.10").add(new BigDecimal("0.20"))    // 0.30, exact
new BigDecimal(0.1)    // 0.1000000000000000055511151231257827... - never do this
```

Q16 Explain the four pillars of OOP using your code.

Encapsulation - Account keeps its balance private and exposes credit() and debit(), so no caller can set a balance directly and skip the rules. **Inheritance** - every entity extends BaseEntity for the auditing columns, and every domain failure extends BusinessException. **Polymorphism** - the global handler catches BusinessException and reads getStatus() and getCode() from whichever subclass was actually thrown, with no instanceof chain. **Abstraction** - services depend on repository interfaces, not on a database, which is what lets the whole business-rule suite run with mocks in two seconds.

Q17 What is the difference between == and equals()?

== compares references for objects - are these the same instance - and values for primitives. equals() compares logical equality as the class defines it. For BigDecimal the distinction bites twice: == is almost never what you want, and equals() compares scale as well as value, so new BigDecimal("100").equals(new BigDecimal("100.00")) is false. Money comparisons in this codebase all use compareTo() == 0.

Q18 How do you implement equals() and hashCode() on a JPA entity?

Carefully, because entity identity changes: before persist the id is null, after persist it is assigned. If hashCode() depends on the id, an entity put into a HashSet before saving becomes unfindable afterwards, because its hash changed while it sat in the bucket.

The approach here is to compare on id but only when it is non-null, and to return a constant-ish hash from Objects.hashCode(id). Using a business key would be better still where one exists.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Account other)) return false;
    return id != null && id.equals(other.id);
}

@Override
public int hashCode() { return Objects.hashCode(id); }
```

Q19 Checked versus unchecked exceptions - which do you use and why?

Everything in this codebase is unchecked. Checked exceptions force every caller in the chain to either catch or declare, which pollutes signatures all the way up for a failure only the top layer can handle. More importantly, **Spring only rolls back a transaction automatically on unchecked exceptions** - a checked exception commits unless you add rollbackFor. For a money system that default is dangerous, so all domain failures extend RuntimeException via BusinessException.

Q20 What are virtual threads and why enable them?

Virtual threads are lightweight threads scheduled by the JVM rather than the OS. A platform thread costs about a megabyte of stack, so a server can hold a few thousand; virtual threads cost a few hundred bytes and you can have millions. When one blocks on I/O it unmounts from its carrier thread instead of parking an OS thread.

This API is mostly blocking on JDBC, which is the ideal case: spring.threads.virtual.enabled gives much higher concurrency with no code changes. The caveat is that they do not help with CPU-bound work, and a database connection pool is still a hard limit - virtual threads do not conjure extra connections.

Q21 What is the difference between an interface and an abstract class?

A class can implement many interfaces but extend only one class. An abstract class can hold state and constructors; an interface holds constants and, since Java 8, default methods but no instance state. The rule of thumb is capability versus identity: an interface says what something can do, an abstract class says what something is. Here BaseEntity is abstract - every entity **is** an auditable thing and inherits four real fields - while repositories are interfaces because Spring Data generates the implementation.

Q22 What is dependency injection and why constructor injection?

Dependency injection means a class declares what it needs and something else supplies it, rather than the class constructing its own collaborators. That is what makes unit testing possible - the test passes mocks instead of real repositories.

Constructor injection specifically, for three reasons: dependencies can be **final**, so the object is immutable and cannot be half-constructed; a missing dependency is a compile-time or startup failure rather than a NullPointerException later; and a constructor with eight parameters is uncomfortable to write, which is useful negative feedback that the class is doing too much. Field injection with @Autowired hides all three signals.

Q23 What does the 'final' keyword do in each position?

On a variable, it cannot be reassigned - note that a final reference to a mutable object can still have its contents changed. On a method, it cannot be overridden. On a class, it cannot be subclassed. All service dependencies here are final fields set in the constructor, which makes them safe to publish across threads.

Q24 What is the difference between String, StringBuilder and StringBuffer?

String is immutable, so concatenation in a loop creates a new object each iteration - $O(n^2)$ for n appends. StringBuilder is a mutable buffer and is the right choice for building strings, as the CSV statement generator does. StringBuffer is the same but synchronised, which is almost never needed and costs performance.

Q25 Explain the Java memory model and where it matters here.

The heap holds objects and is shared across threads; each thread has its own stack for frames and local variables. The part that matters for a concurrent system is visibility: without synchronisation, one thread's write to a shared field is not guaranteed to be visible to another.

This application deliberately does not rely on in-memory synchronisation for correctness, though. The shared state is the account balance, and it lives in the database - so the coordination mechanism is a database row lock, not a Java lock. That is the right call for a system with more than one instance, where a synchronized block on one JVM means nothing to another.

Q26 What is the Optional type and how do you use it properly?

Optional expresses 'a value may legitimately be absent' in the type system, replacing a null return that callers forget to check. The idiom used throughout the repositories here is to return Optional from a finder and let the caller map or throw:

The anti-patterns are calling get() without checking, using Optional as a field or a method parameter, and returning Optional from something that should just throw.

```
return accountRepository.findByAccountNumber(accountNumber)
    .orElseThrow(() -> ResourceNotFoundException.of("Account", accountNumber));
```

CHAPTER 3

Spring Boot

The framework, its magic, and where the magic stops

3

Q27 What is Spring Boot and what problem does it solve?

Spring Boot is Spring with opinionated defaults. Plain Spring is powerful but requires extensive XML or Java configuration, an external servlet container, and manual dependency version alignment. Boot adds auto-configuration, starter dependencies that pull a version-compatible set, an embedded server so the application is a runnable jar, externalised configuration, and production endpoints through Actuator. The result is that this project's configuration is one YAML file and a handful of `@Configuration` classes.

Q28 How does auto-configuration actually work?

`@SpringBootApplication` includes `@EnableAutoConfiguration`, which asks Spring to load auto-configuration classes listed in `META-INF/spring/...AutoConfiguration.imports` inside each starter jar. Each of those classes is guarded by `@Conditional` annotations - `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`.

So `DataSourceAutoConfiguration` only fires if a JDBC driver is on the classpath and you have not already defined a `DataSource` yourself. That last condition is the important one: auto-configuration always backs off when you supply your own bean, which is why defining a `SecurityFilterChain` here silently disables Boot's default one. Run with `--debug` to see the full report of what matched and what did not.

Q29 What is the Spring IoC container and the bean lifecycle?

The container creates, configures and wires the objects the application is made of. Inversion of control means the framework calls you rather than you calling it: you declare a bean and the container decides when to build it and what to inject.

The lifecycle is: instantiate, populate dependencies, run `*Aware` callbacks, run `BeanPostProcessors` before initialisation, run `@PostConstruct` and `InitializingBean`, run `BeanPostProcessors` after initialisation - which is where AOP proxies such as `@Transactional` are created - then the bean is ready, and finally `@PreDestroy` on shutdown.

Q30 Explain the bean scopes.

Singleton is the default: one instance per container, which is why beans must be stateless. **Prototype**: a new instance per injection. **Request**, **session** and **application** exist for web applications. Everything in this project is a singleton, and that is deliberate - the services hold only their injected collaborators, never per-request state, so they are safe to share across all concurrent requests.

Q31 What is the difference between `@Component`, `@Service`, `@Repository` and `@Controller`?

Technically almost nothing - `@Service`, `@Repository` and `@Controller` are all meta-annotated with `@Component`, so all four are picked up by component scanning. The difference is intent, which matters for a reader, plus two concrete behaviours: `@Repository` enables exception translation, converting vendor-specific `SQLExceptions` into Spring's `DataAccessException` hierarchy, and `@Controller` / `@RestController` hook into the MVC handler mapping.

Q32 How does @RestController differ from @Controller?

@RestController is @Controller plus @ResponseBody on every method, so return values are serialised straight into the response body by an `HttpMessageConverter` - Jackson, for JSON - rather than being resolved as a view name. For an API that serves JSON to a separate frontend, it removes an annotation from every single method.

Q33 How does configuration binding work, and why @ConfigurationProperties over @Value?

@Value injects a single property with a SpEL expression. @ConfigurationProperties binds a whole tree onto a typed object, validated at startup. This project binds the `app.*` block onto an `AppProperties` record, so a typo in a property name is a startup failure rather than a runtime surprise, related settings are grouped, and IDE autocomplete works. It also keeps every tunable overridable by an environment variable, which is what lets the same jar run in dev, Docker and production without a rebuild.

```
@ConfigurationProperties(prefix = "app")
public record AppProperties(Jwt jwt, Cors cors, Limits limits) {
    public record Jwt(String secret, long accessTokenMinutes,
                      long refreshTokenDays, String issuer) {}
}
```

Q34 What are Spring profiles and how do you use them?

A profile is a named set of configuration active under a condition. Here the default profile uses in-memory H2 so the API runs with nothing installed, and the 'postgres' profile - activated by Docker Compose through `SPRING_PROFILES_ACTIVE` - swaps in the PostgreSQL datasource, dialect and pool settings. Beans can also be annotated @Profile to exist only in certain environments.

Q35 How does @RestControllerAdvice work?

It is @ControllerAdvice plus @ResponseBody: a class whose @ExceptionHandler methods apply across all controllers. When a controller method throws, Spring looks for the most specific handler for that exception type and invokes it, and the return value is serialised as the response.

It is why no controller in this project contains a try/catch, and why every failure - validation, domain, security, database, unexpected - comes back in the same `ApiError` shape with a machine-readable code the React client can branch on.

Q36 What is Spring AOP and where does this project rely on it?

Aspect-oriented programming factors out cross-cutting concerns into aspects applied around methods. Spring implements it with runtime proxies - JDK dynamic proxies for interfaces, CGLIB subclasses otherwise.

This project relies on it constantly without writing an aspect: @Transactional is an aspect that opens and commits a transaction, @PreAuthorize is an aspect that checks authority, and @Scheduled is driven by the same machinery. Understanding that these are proxies is what explains the self-invocation problem - a call through 'this' never passes the proxy, so the advice never runs.

Q37 What is Spring Boot Actuator and what do you expose?

Actuator adds production endpoints: health, info, metrics, and more. This project exposes only health, info and metrics, and health details only to authorised callers - exposing everything publicly leaks environment variables and internal state. The health endpoint is what the Docker healthcheck polls, and what Compose waits on before starting the frontend container.

Q38 How does @Valid trigger validation, and what happens on failure?

@Valid on a controller parameter tells Spring to run Jakarta Bean Validation on the bound object before the method body executes. Hibernate Validator evaluates the constraint annotations on the DTO. On failure Spring throws MethodArgumentNotValidException, which the global handler converts into a 400 with a per-field breakdown so the React form can highlight exactly which input was wrong.

@Validated is the variant for class-level validation of path variables and request parameters, and it throws ConstraintViolationException instead - which is why the handler handles both.

Q39 What is the difference between @RequestParam, @PathVariable and @RequestBody?

@PathVariable reads a segment of the URL path and is right for identifying a resource - /accounts/{accountNumber}. @RequestParam reads a query-string parameter and is right for filtering, sorting and paging - ?type=DEPOSIT&page=2. @RequestBody deserialises the request body, normally JSON, into an object and is used for creates and updates.

Q40 How do you handle CORS, and why is it needed at all?

The browser's same-origin policy blocks a page served from localhost:5173 from reading a response from localhost:8080. CORS is the server telling the browser which origins may. It is configured here as an allow-list in CorsConfigurationSource, driven by a property rather than a wildcard.

In development it is not even needed: Vite proxies /api to the backend, so from the browser's point of view everything is same-origin. In production nginx does the same. CORS matters mainly for the case where the frontend is served from a genuinely different origin.

Q41 What does @Scheduled do and what are its pitfalls?

It runs a method on a cron expression or fixed delay, once @EnableScheduling is present. The interest job here runs at 01:00 on the first of each month.

Two pitfalls. First, the scheduled method must not call a @Transactional method on itself - self-invocation bypasses the proxy, so the transaction never starts. That is exactly why InterestScheduler is a separate bean from TransactionService. Second, with multiple instances every instance runs the job, so a real deployment needs a distributed lock such as ShedLock.

Q42 How would you add caching?

@EnableCaching plus @Cacheable on read methods, with a real cache manager - Caffeine locally or Redis when there is more than one instance. The dashboard aggregates are the obvious candidate: they are expensive, read often and tolerate being a minute stale.

What I would **not** cache is anything on the money path. A cached balance that is thirty seconds old is exactly the lost-update problem all over again, just with a different mechanism.

CHAPTER 4

JPA and Hibernate

Persistence, mapping, and the traps

4

Q43 What is the difference between JPA, Hibernate and Spring Data JPA?

JPA is a specification - a set of interfaces and annotations, with no implementation. **Hibernate** is the most common implementation of that specification, and adds features beyond it. **Spring Data JPA** sits on top of a JPA provider and generates repository implementations from interfaces, so you declare `findByEmailIgnoreCase` and never write the query. Three layers, each solving a different problem.

Q44 Explain the JPA entity lifecycle states.

Transient - a new object the persistence context has never seen. **Managed** - attached to the persistence context; Hibernate tracks changes and will flush them at commit without you calling `save`. **Detached** - was managed, but the context closed. **Removed** - marked for deletion at flush.

The managed state is why `TransactionService` can call `account.debit(amount)` and never call `save()`: the account was loaded inside the transaction, so it is managed, and dirty checking writes the `UPDATE` at commit.

Q45 What is the persistence context and what is dirty checking?

The persistence context is a first-level cache scoped to the transaction. It guarantees that loading the same row twice returns the same object instance, and it keeps a snapshot of each entity as loaded. At flush, Hibernate compares each managed entity against its snapshot and issues `UPDATES` only for what actually changed. That is dirty checking, and it is why explicit `save` calls are usually unnecessary inside a transaction.

Q46 What is the N+1 select problem and how did you solve it here?

You load `N` entities in one query, then touch a lazy association on each one, firing `N` more queries. Rendering a 25-row statement page would issue 1 query for the transactions and 25 more to resolve each row's account - 26 round trips for one screen.

The fix used here is `JOIN FETCH` in the repository query, which loads the associations in the same statement. Note the separate `countQuery`: a paged query with `JOIN FETCH` cannot be counted directly, so Spring Data needs the count spelled out. Other options are an `@EntityGraph` or batch fetching.

```
@Query(value = ""
    SELECT t FROM Transaction t
    JOIN FETCH t.account a
    JOIN FETCH a.owner
    LEFT JOIN FETCH t.counterpartyAccount
    WHERE a.id IN :accountIds ...
    """,
    countQuery = "SELECT COUNT(t) FROM Transaction t WHERE ...")
Page<Transaction> search(...);
```

Q47 FetchType.LAZY or EAGER - which and why?

LAZY for every association in this codebase. EAGER means every load of a Transaction also loads its Account, that account's owner, and anything eager on those - a cascade of joins you did not ask for, on every single query, whether or not you need the data.

LAZY is the correct default, and where a specific query does need the association you ask for it explicitly with JOIN FETCH. The cost of LAZY is LazyInitializationException if you touch the association outside the transaction, which is exactly why open-in-view is disabled and why entities are mapped to DTOs inside the service.

Q48 What is open-in-view and why did you disable it?

Open Session In View keeps the persistence context open for the whole HTTP request, including view rendering, so lazy associations still resolve in the controller. Spring Boot enables it by default and warns about it at startup.

I disabled it because it hides N+1 problems rather than preventing them - queries fire silently during serialisation, outside any transaction you can see - and it holds a database connection for the entire request, including time spent writing bytes to a slow client. With it off, a missing fetch fails loudly in development instead of quietly degrading in production.

Q49 How do you write custom queries in Spring Data JPA?

Four ways, in order of preference. **Derived queries** from the method name - findByOwnerIdOrderByCreatedAtDesc - which Spring parses into a query. **@Query with JPQL**, for anything the naming scheme cannot express, which is what the statement search uses. **Specifications or Criteria API** for dynamically composed predicates. **Native SQL** as a last resort, accepting that it ties you to one database.

This project uses derived queries where they are readable and @Query where the filter combination would produce an unreadable method name.

Q50 What is the difference between JPQL and native SQL?

JPQL queries the **entity model** - class and field names - and the provider translates it into SQL for the configured dialect, so it is portable across databases. Native SQL is the real thing, with full access to vendor features, at the cost of portability. Everything here is JPQL except the one deliberate exception in the demo seeder, which uses a JdbcTemplate update to backdate timestamps that JPA auditing would otherwise overwrite.

Q51 How does pagination work and why not return Page directly?

You pass a Pageable - page number, size and sort - and Spring Data returns a Page containing the content plus total counts, running a second COUNT query to get them.

I map Page onto my own PageResponse record rather than returning it. Spring's Page serialises with an unstable, implementation-detail-heavy JSON shape that has changed between versions; exposing it makes the framework's internals part of my public API contract. Page sizes are also capped server-side, so a client cannot request a million rows.

Q52 What is @Version and how does optimistic locking work?

@Version marks a column Hibernate increments on every update. Each UPDATE carries the version it read in its WHERE clause - so if another transaction committed first, zero rows match, and Hibernate raises OptimisticLockingFailureException.

It takes no locks, so it costs nothing when conflicts are rare, and it fails at commit time when they are not. In this project it is a second guard behind the pessimistic lock: it catches any future write path that forgets to take the lock.

```
UPDATE accounts SET balance = ?, version = 6
WHERE id = ? AND version = 5;
-- 0 rows updated -> someone else got there first -> throw
```

Q53 What is @MappedSuperclass and how is it different from @Entity?

@MappedSuperclass provides mapped fields to subclasses without being an entity itself - there is no table for it and you cannot query it. BaseEntity here carries createdAt, updatedAt, createdBy and updatedBy, and every entity inherits those four columns into its own table. Entity inheritance strategies - SINGLE_TABLE, JOINED, TABLE_PER_CLASS - are a different tool, for when the subclasses really are queryable types.

Q54 How does JPA auditing work, and where did it bite you?

@EnableJpaAuditing plus AuditingEntityListener populates @CreatedDate, @LastModifiedDate, @CreatedBy and @LastModifiedBy automatically, with an AuditorAware bean supplying the current user - falling back to 'system' for background jobs.

It bit me in the demo seeder. I wanted seven months of backdated history so the dashboard charts had shape, but auditing stamps @CreatedDate **unconditionally** on insert, collapsing everything onto today and leaving the charts flat. The fix was to insert normally and then issue a direct UPDATE to rewrite created_at - contained in the seeder, leaving auditing intact for all real writes.

Q55 What does cascade do, and why is CascadeType.ALL risky?

Cascade propagates an operation from a parent to its associated entities. ALL includes REMOVE, which means deleting a parent silently deletes children - fine for a genuinely owned composition, dangerous otherwise. Deleting an Account with cascade ALL would destroy its transaction history, which is precisely the data a bank must never lose. There is no cascade on that association here; closing an account is a status change, not a delete.

Q56 What is the difference between save() and saveAndFlush()?

save() makes the entity managed and schedules the insert or update; the SQL is issued at flush time, which is normally at commit. saveAndFlush() forces the SQL immediately. Flushing early matters when you need a generated id right away, or when a later query in the same transaction must see the row. The seeder uses flush() before backdating, because the UPDATE needs the ids to exist.

Q57 What is a DTO projection and when would you use one?

A query that selects only the columns you need, into an interface or a constructor expression, rather than hydrating full entities. It is faster and avoids loading data you will discard.

The dashboard aggregates here return Object[] tuples from grouped queries rather than entity lists - the same idea. For a larger system I would use typed record projections to keep it readable.

Q58 Why did you choose ddl-auto: validate?

Because Flyway owns the schema. ddl-auto: update looks convenient and works in development, but in production it issues ALTER statements nobody reviewed, it will never drop or rename anything so mistakes accumulate forever, and there is no way to roll it back or reproduce the same schema elsewhere.

validate does one useful thing: it fails fast at startup if an entity and the migrated schema have drifted apart. That converts a class of silent production incidents into a loud failure on a developer's laptop.

CHAPTER 5

Database and SQL

Schema design, indexing, and query behaviour

5

Q59 Walk me through your schema design.

Six tables. **users** holds identity and credentials, with a `public_id` UUID exposed to clients so the numeric primary key never leaves the server. **accounts** belongs to a user and holds the balance as `NUMERIC(19,4)` plus a version column. **transactions** is the append-only ledger, where each row belongs to one account and optionally names a counterparty, and where a transfer's two legs share one reference. **beneficiaries** are saved payees, unique per owner and account number. **refresh_tokens** stores SHA-256 hashes so sessions are revocable. **audit_logs** is append-only and deliberately has no foreign keys.

Q60 Why does audit_logs have no foreign key to users?

Because an audit row must outlive whatever it describes. If a user is deleted, the record that they attempted a transfer must survive - that is the entire point of an audit trail. So the actor is stored as text rather than as a reference that could cascade away or block the delete.

Q61 What indexes did you create and why each one?

Every index answers a specific query. **(account_id, created_at DESC)** matches the statement query's WHERE and ORDER BY exactly, so paging through history is an index range scan rather than a full scan plus sort. **UNIQUE (idempotency_key)** is not for speed at all - it is the correctness guarantee that two concurrent retries cannot both post. **account_number** is unique and indexed because the API addresses accounts by number, so it is on the hot path of every transfer. **(owner_id, account_number)** on beneficiaries prevents duplicate payees. And `created_at` plus `action` on `audit_logs` back the admin screen's default ordering and filter, on a table that only ever grows.

Q62 What are the downsides of adding indexes?

Every index is a second data structure that must be updated on every INSERT, UPDATE and DELETE, so writes get slower and storage grows. On an append-only ledger that is a real cost. The rule I applied was one index per query pattern that actually exists, not one per column that might be filtered someday.

Q63 Explain the ACID properties with an example from your system.

Atomicity - a transfer's debit and credit both commit or neither does; there is no state where money left one account without arriving at the other. **Consistency** - CHECK constraints and foreign keys mean a committed transaction always leaves valid data; a negative amount is unrepresentable. **Isolation** - concurrent transfers do not see each other's uncommitted work, enforced here with `SELECT ... FOR UPDATE`. **Durability** - once committed, the write survives a crash, because the database has written it to a durable log.

Q64 What are the SQL isolation levels and which do you use?

READ UNCOMMITTED allows dirty reads - seeing uncommitted data. **READ COMMITTED** prevents those but allows non-repeatable reads. **REPEATABLE READ** prevents those but classically allows phantom rows. **SERIALIZABLE** prevents all three at the highest cost.

I use READ COMMITTED - the default - and rely on explicit row locks. I originally set REPEATABLE READ on the transfer and then removed it: the FOR UPDATE lock already prevents the balance changing underneath the transaction, so the higher level added no safety while making the database far more likely to abort with a serialization failure under load. Being able to explain **why I removed it** is more useful than having set it in the first place.

Q65 What is the difference between pessimistic and optimistic locking?

Pessimistic assumes conflict is likely and locks up front - SELECT ... FOR UPDATE - so other transactions block. **Optimistic** assumes conflict is rare, takes no locks, and detects the collision at commit through a version column, failing the loser.

Pessimistic is right for hot rows and where a late failure is expensive. Money movement here uses it because an active customer's account genuinely does see concurrent writes, and telling someone their transfer failed after they pressed Send is much worse than making them wait a few milliseconds.

Q66 What causes a deadlock and how do you prevent it?

A circular wait: A holds a lock B wants while B holds a lock A wants. In this system that is Priya transferring to Rahul at the same instant Rahul transfers to Priya, if each locks its own source first.

The prevention is to remove the cycle by imposing a total order on lock acquisition. Both accounts are sorted by account number and locked lowest-first, regardless of direction. Then both transactions want the same row first: one wins, the other waits, and both complete. It makes a deadlock structurally impossible rather than merely unlikely - and there is an integration test that would hang if I got it wrong.

Q67 Why NUMERIC(19,4) rather than FLOAT or an integer number of paise?

FLOAT is out because binary floating point cannot represent decimal fractions exactly and the error accumulates. Integer paise is a legitimate alternative that some systems use - it is exact and fast - but it pushes the scaling into application code, where every read and write must remember to divide or multiply by 100, and one forgotten conversion is a hundredfold error.

NUMERIC keeps the decimal semantics in the database where the constraint can be expressed. Scale 4 rather than 2 leaves room for intermediate values such as accrued interest before they are rounded to settlement precision.

Q68 How do database migrations work here and why not auto-DDL?

Flyway runs versioned SQL files at startup, records each in a schema_history table, and applies only what has not run. That makes the schema a reviewable artefact in version control with a full history, reproducible on any machine.

The migration is written in portable SQL - no PostgreSQL ENUM types, no SERIAL, enums as VARCHAR with CHECK constraints, identity columns in the SQL:2003 form - so the identical file runs on both PostgreSQL and the H2 used for local development and tests. One schema definition, two engines.

Q69 How would you find and fix a slow query?

Start with evidence, not guesses. Turn on slow-query logging to find which query is actually slow, then run EXPLAIN ANALYZE on it and look for sequential scans on large tables, sorts that spill to disk, and a large gap between estimated and actual row counts, which usually means stale statistics.

The fixes, in order: add or fix an index so the WHERE and ORDER BY are covered; rewrite the query to be sargable; reduce the data returned; and only then consider denormalising. balance_after in the ledger is exactly that last step, taken deliberately - without it, every statement row would need a running sum over the account's whole history.

Q70 When would you denormalise, and where did you?

When a read is frequent and expensive and the derived value can be maintained reliably at write time. `balance_after` on each transaction is the example: it is written once, inside the same transaction that changed the balance, so it cannot drift, and it turns statement rendering from a running aggregate into a plain column read. The cost is that it is redundant data - which is acceptable precisely because ledger rows are immutable and can never be updated out of sync.

CHAPTER 6

Security and JWT

Authentication, authorization, and the threat model

6

Q71 Explain your authentication flow end to end.

The client POSTs email and password to /auth/login. AuthenticationManager delegates to DaoAuthenticationProvider, which loads the user through UserDetailsService and compares the password with BCrypt. On success the server issues two tokens: a 15-minute access token and a 7-day refresh token, and stores a SHA-256 hash of the refresh token so it can be revoked.

The client stores both and sends the access token as an Authorization: Bearer header on every request. JwtAuthenticationFilter verifies the signature, issuer and expiry, confirms the token is an access token rather than a refresh token, loads the user and populates the SecurityContext. When the access token expires, an axios interceptor silently exchanges the refresh token for a new pair and replays the failed request.

Q72 What is a JWT and what are its three parts?

A JSON Web Token is a signed, self-contained token. Base64url header, payload and signature, separated by dots. The header names the algorithm; the payload holds claims - here subject, issuer, issued-at, expiry, role, name and a token-type claim; the signature is an HMAC over the first two parts using the server's secret.

The critical point is that the payload is **encoded, not encrypted** - anyone can decode and read it. The signature only proves it has not been tampered with. So a JWT must never carry anything secret.

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJwcm15YUBub3ZhYmFuay5pbyIsInJvbmGUiOiJDVFNUT01FUlJ9 . NQJ3VNYE...
```

header (alg) payload (claims - readable by anyone) signature

Q73 Why two tokens instead of one long-lived token?

It resolves a genuine tension. A JWT is verified without a database lookup, which is what makes it scale - and also what makes it impossible to un-issue. A single long-lived token would therefore be convenient and unrevocable, which is a bad combination.

Splitting them means the token sent on every request is short-lived and disposable, so a leaked one has a small window. The token that grants long sessions is stored server-side and therefore revocable. Logout, a password change and an admin disabling an account all work by revoking refresh tokens.

Q74 What is refresh-token rotation and why does it matter?

Every refresh consumes the old token and issues a new pair. If an attacker steals a refresh token and uses it, the legitimate user's copy is already revoked, so their next refresh fails and they are forced to log in again. The theft becomes **visible** instead of granting silent, indefinite access. A more advanced version detects reuse of an already-rotated token and revokes the whole family.

Q75 Why store only a hash of the refresh token?

For the same reason passwords are hashed: a database leak must not hand out working sessions. Plain SHA-256 with no salt is correct here, unlike for passwords - the input is a signed JWT with well over 200 bits of entropy, so there is nothing to brute-force and a rainbow table is meaningless. Password hashing needs BCrypt precisely because human passwords are low-entropy.

Q76 Explain BCrypt. Why not SHA-256 for passwords?

SHA-256 is designed to be fast, which is exactly wrong for passwords - a GPU can compute billions per second, so a stolen table falls quickly. BCrypt is deliberately slow, with a tunable cost factor: strength 10 means 2^{10} key-expansion rounds, and each increment doubles the work.

It also generates a random salt per password and stores it inside the hash string, so two users with the same password get different hashes and precomputed rainbow tables are useless. Argon2 is the modern recommendation and is what I would use next.

```
$2a$10$N9qo8uL0ickgx2ZMRZoMye IjZAgcfl7p92ldGxad68LJZdL17lhWy
|   |   |                       |
|   |   +-- 22-char salt        +-- 31-char hash
|   +-- cost factor 10
+-- algorithm version
```

Q77 How does the Spring Security filter chain work?

It is an ordered chain of servlet filters, each with one job. A request passes through CORS handling, then my JwtAuthenticationFilter which turns a bearer token into a populated SecurityContext, then the AuthorizationFilter which applies the route rules from SecurityConfig. If nothing rejects it, the request reaches the controller.

My filter extends OncePerRequestFilter so it does not repeat on forwards or async dispatches, and it never throws on a bad token - it simply does not authenticate, leaving the authorization rules to produce a clean 401.

Q78 Authentication versus authorization - and where does each happen?

Authentication answers 'who are you', authorization answers 'what may you do'. Authentication happens once per request in JwtAuthenticationFilter. Authorization happens in three places here, deliberately: route rules in SecurityConfig (/admin/** requires ADMIN), @PreAuthorize on admin service methods, and ownership checks inside the services.

The third is the one people forget. Route-level rules confirm you are a customer; only the service check confirms you are the customer who owns **this** account.

Q79 What is IDOR and how do you prevent it?

Insecure Direct Object Reference: an authenticated user changes an identifier in a URL and reads someone else's data. Every account lookup here goes through requireOwnedAccount(), which loads the account and verifies the caller owns it - or is an admin.

It returns **404**, not **403**, and that is deliberate. A 403 confirms the account number exists, which turns the endpoint into an enumeration oracle. A 404 is indistinguishable from a number that was never issued. There are integration tests asserting exactly this.

```
Account account = accountRepository.findByAccountNumber(accountNumber)
    .orElseThrow(() -> ResourceNotFoundException.of("Account", accountNumber));

if (!SecurityUtils.isAdmin() && !account.getOwner().getId().equals(current.getId())) {
    auditService.failure("UNAUTHORISED_ACCOUNT_ACCESS", ...);
    throw ResourceNotFoundException.of("Account", accountNumber); // 404, not 403
}
```

Q80 Why did you disable CSRF? Isn't that a vulnerability?

Not here, and the reasoning matters. CSRF works because browsers attach **cookies** automatically to cross-site requests, so a malicious page can trigger an authenticated action. This API authenticates with an Authorization header, which a cross-site form or image tag cannot set - the browser will not add it, and CORS blocks the JavaScript that could.

So CSRF tokens would add machinery that protects against nothing. The moment I move tokens into cookies - which I would like to, for XSS reasons - CSRF protection has to come straight back. It is a trade, not a free win.

Q81 Can a user register themselves as an admin?

No, and there are two independent reasons. RegisterRequest is a record with no role field, so the JSON simply has nowhere to bind - sending "role":"ADMIN" is silently ignored. And AuthService hard-codes Role.CUSTOMER when constructing the user. Role changes are an admin-only operation. There is a test that posts a role field and asserts the created user is still a CUSTOMER.

Q82 How do you prevent user enumeration on the login endpoint?

Two ways. The response for a wrong password and for an unknown email is byte-for-byte identical - one generic 'Invalid email or password' with the same status and code. And hideUserNotFoundExceptions is enabled on the DaoAuthenticationProvider, so the password comparison still runs for an unknown user and the timing does not give the answer away either. There is a test asserting both messages are equal.

Q83 Where do you store tokens on the client, and what is the trade-off?

localStorage. It is simple and survives a refresh, but it is readable by any JavaScript on the page, so a successful XSS attack steals the session. The alternative is an httpOnly, Secure, SameSite=Strict cookie, which JavaScript cannot read - at the cost of needing CSRF protection back.

I would move to cookies for a real deployment. Naming the trade-off honestly is better than pretending localStorage is fine.

Q84 How do you prevent SQL injection?

Everything goes through JPA with bound parameters - :accountNumber, not string concatenation. The database receives the query structure and the values separately, so a value can never be parsed as SQL. There is no place in the codebase where user input is concatenated into a query. If I ever needed native SQL, it would still use bound parameters.

Q85 What does @PreAuthorize do and why use it when routes are already secured?

@PreAuthorize evaluates a SpEL expression before the method runs, throwing AccessDeniedException if it is false. AdminService carries hasRole('ADMIN') at class level even though /api/v1/admin/** is already locked down in SecurityConfig.

That duplication is defence in depth. If someone later exposes one of those methods through a different URL, or calls it from a new controller, the authorization travels with the method rather than with the route it happened to sit behind.

Q86 What would you add to harden this further?

Rate limiting and account lockout on the auth endpoints - Bucket4j would be the quickest path. Two-factor authentication, with step-up for high-value transfers. Tokens in httpOnly cookies. A Content-Security-Policy header to reduce XSS impact. Secrets from a manager rather than environment variables. Dependency scanning in CI. And structured security logging shipped somewhere that alerts, since audit records are only useful if someone reads them.

CHAPTER 7

Concurrency and Transactions

The heart of the system - expect the most questions here

7

Q87 What is a lost update and how does your system prevent it?

Two transactions read the same balance, both decide a withdrawal is affordable, and both write - one silently overwrites the other. Start with 1000, two withdrawals of 700, and the account ends at 300 having paid out 1400. The bank created 700 out of nothing.

The prevention is to serialise the read-decide-write sequence with a pessimistic row lock. The balance is read with `SELECT ... FOR UPDATE`, so the second transaction blocks until the first commits and then re-reads the balance the first one actually wrote - at which point it correctly refuses.

TIME	Transaction A	Transaction B	BALANCE
t1	SELECT ... FOR UPDATE -> 1000		1000
t2		FOR UPDATE ... BLOCKED	1000
t3	1000 >= 700, approve	(waiting)	1000
t4	write 300, COMMIT	(waiting)	300
t5		unblocks, re-reads -> 300	300
t6		300 >= 700? NO -> reject	300

Q88 Why pessimistic locking here rather than optimistic?

Because conflicts on a hot account are genuinely likely, and because a late failure is expensive. Optimistic locking lets both transactions do all their work and then fails one at commit - which for a user means being told their transfer failed *after* they pressed Send. Pessimistic locking makes the second transaction wait a few milliseconds and then succeed or fail on accurate data.

Optimistic is still there as a `@Version` column, as a second guard against any write path that forgets to take the lock.

Q89 How does SELECT ... FOR UPDATE work?

It takes an exclusive row-level lock on every row the query returns, held until the transaction commits or rolls back. Any other transaction that tries to lock or update those rows blocks until then. In JPA it is expressed with `@Lock(LockModeType.PESSIMISTIC_WRITE)` on the repository method.

It must be inside a transaction to mean anything - the lock's lifetime *is* the transaction's lifetime.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.accountNumber = :accountNumber")
Optional<Account> findByAccountNumberForUpdate(@Param("accountNumber") String accountNumber);
```

Q90 Your transfer locks two rows. How do you avoid a deadlock?

By locking in a globally consistent order. If each transaction locked its own source account first, then Priya paying Rahul at the same moment Rahul pays Priya would produce a circular wait - each holding one row, each waiting for the other, forever.

Instead both accounts are sorted by account number and the lower one is locked first, regardless of which is the sender. Now both transactions want the same row first: one acquires both locks and finishes, the other waits and then

proceeds. The cycle is structurally impossible.

It is verified, not assumed - an integration test fires 20 transfers in both directions simultaneously and asserts both balances return exactly to their starting values. It would hang and time out if the ordering were wrong.

```
String first = min(request.fromAccountNumber(), request.toAccountNumber());
String second = max(request.fromAccountNumber(), request.toAccountNumber());

Account firstLocked = lockAccount(first); // always the lower number
Account secondLocked = lockAccount(second);

Account source = firstLocked.getAccountNumber().equals(request.fromAccountNumber())
    ? firstLocked : secondLocked;
Account destination = source == firstLocked ? secondLocked : firstLocked;
```

Q91 How does @Transactional work under the hood?

Spring creates a proxy around the bean. When an annotated method is called through that proxy, the transaction interceptor opens a transaction (or joins one), invokes the method, and then commits on normal return or rolls back on an unchecked exception. The connection is bound to the thread, so every repository call inside the method uses the same one.

Two consequences follow from it being a proxy, and both appear in this codebase: self-invocation does not work, and the annotation is ignored on non-public methods.

Q92 What is the self-invocation problem? Did you hit it?

Calling a @Transactional method from another method of the **same class** goes through 'this', not the proxy, so the annotation is silently ignored - no transaction is started, and nothing warns you.

Yes, twice. The monthly interest job originally had a scheduled method looping over accounts and calling a @Transactional method on itself, which would have run the entire batch with no transaction at all. I moved it into a separate InterestScheduler bean. The same reasoning drove MoneyMovementFacade: the retry has to start a new transaction, so it has to cross a proxy boundary, so it has to be a different bean.

Q93 What is transaction propagation and where do you use a non-default one?

Propagation decides what happens when a transactional method is called from inside an existing transaction. REQUIRED, the default, joins it. REQUIRES_NEW suspends it and starts a fresh one.

AuditService.record() uses REQUIRES_NEW. That is deliberate: if a transfer is rejected for insufficient funds and its transaction rolls back, the record that the attempt was made must still survive. A failed attempt is exactly what a compliance officer wants to see - if the audit joined the rolled-back transaction, the evidence would vanish with it.

Q94 What is the cost of REQUIRES_NEW?

A suspended transaction still holds its database connection while the inner one takes a second. So a money-movement request occupies two pool connections at its peak.

With a small pool and enough concurrency, that is a real hazard: requests holding row locks could end up queuing for a connection that only a lock-holder can release - a self-inflicted deadlock that has nothing to do with the database's own locking. The pool is sized at 24 with leak detection enabled for exactly this reason, and it is why I would move audit writes onto an async queue if throughput mattered.

Q95 You said concurrent transfers returned 500s. Explain what was happening.

Under contention a database will sometimes refuse a transaction rather than queue it - a lock-wait timeout, or being selected as a deadlock victim. H2 raises SQLSTATE 40001 for this. Spring translates it into `CannotAcquireLockException`, which my catch-all handler turned into a 500.

Two things were wrong. A 500 is the wrong status - it implies server breakage and that the request might have partly succeeded, when in fact nothing was applied and retrying is safe. And the application should have retried rather than surfacing a transient failure to the user at all.

The fix was a bounded retry with jittered exponential backoff in a bean outside the transaction boundary, plus mapping `ConcurrencyFailureException` to a retryable 409. Twelve concurrent transfers went from 3 succeeding to 12.

Q96 Why does the retry need jitter?

Without randomness, every loser of a collision sleeps the same amount and wakes at the same instant - so they collide again, and again. Jitter spreads the retries out, turning one contended moment into a resolved one rather than a repeating one. It is the same reasoning behind randomised exponential backoff in network protocols.

```
long ceiling = BASE_BACKOFF_MILLIS * (1L << (attempt - 1));
long delay = ThreadLocalRandom.current()
    .nextLong(BASE_BACKOFF_MILLIS, ceiling + BASE_BACKOFF_MILLIS);
```

Q97 Which exceptions do you retry, and which do you not?

Only `ConcurrencyFailureException` and its subclasses - optimistic-lock failures, pessimistic-lock timeouts and deadlock-victim aborts. They share that Spring superclass precisely because the remedy is the same for all three: nothing was applied, try again.

Business failures - insufficient funds, a frozen account, a breached daily limit - are `BusinessExceptions` and propagate on the first attempt. Retrying them would just fail four times more slowly and delay the user's error message.

Q98 What is idempotency and how did you implement it?

An idempotent operation produces the same result whether applied once or many times. It matters because networks are unreliable: a client that sends a transfer and never receives a response cannot know whether it was applied, so it retries - and without protection the money moves twice.

The client generates a UUID per transfer attempt and sends it as `idempotencyKey`. The server checks it before doing any work and returns the original result if it has been seen. Critically, the column also carries a **UNIQUE index**, because two concurrent retries could both pass the application-level check in the same microsecond. The lookup is the fast path; the constraint is the guarantee.

Q99 Where should the idempotency key come from - client or server?

The client, and it must be stable across retries of the same logical operation. If the server generated it, every retry would get a new one and it would protect nothing. In the React app the key is minted once per **form session** - when the form resets - not per click, so a double-click reuses it while a genuinely new transfer gets a fresh one.

Q100 Which HTTP methods are naturally idempotent?

GET, PUT and DELETE are, by definition in the HTTP spec: repeating them has the same effect as doing them once. POST is not, which is exactly why a POST that moves money needs an explicit idempotency key. This is also why a transfer is a POST rather than a PUT - it creates a new ledger entry each time it legitimately runs, rather than setting a resource to a known state.

Q101 How would you test for a race condition?

Not with a sequential test - it can never catch one. TransferConcurrencyIT creates 20 threads, holds them all at a CountdownLatch, releases them at the same instant to maximise real contention, and then asserts the **invariant** rather than the implementation: the sum of both balances is unchanged, the sender dropped by exactly the total moved, and exactly 40 ledger rows exist for 20 transfers.

One detail that matters: the test class is deliberately not @Transactional. A transactional test would roll everything into a single transaction and the worker threads would never see each other's data - which is precisely the interaction under test.

Q102 If you had two API instances, would your locking still work?

Yes, and that is the point of putting the lock in the database rather than in Java. A synchronized block or a ReentrantLock only coordinates threads inside one JVM, so a second instance would know nothing about it and the lost update would come straight back. A row lock is held by the database, which both instances share, so it coordinates across all of them. The same reasoning applies to scheduled jobs, which is why a multi-instance deployment would need a distributed lock such as ShedLock for the interest run.

CHAPTER 8

REST API Design

Contracts, status codes, and versioning

8

Q103 What makes an API RESTful?

Resources identified by URIs, manipulated with the standard HTTP verbs, with a stateless server - each request carries everything needed to process it. Representations are usually JSON, responses declare their cacheability, and the interface is uniform across resources.

This API is stateless in the strict sense: no HttpSession, no server-side session store, identity carried in a token on every request. That is what lets instances scale horizontally without sticky sessions. It is not fully HATEOAS-driven - most APIs are not, and I would not claim otherwise.

Q104 How did you choose your status codes?

200 for a successful read, 201 for something created, 204 for a successful action with no body. 400 when the request is malformed or fails bean validation. 401 when authentication is missing or invalid, 403 when the caller is authenticated but not permitted. 404 for a missing resource - and, deliberately, for a resource that exists but belongs to someone else. 409 for a conflict, including lock contention that survived the retries. 422 when the request is well-formed but the domain refuses it - insufficient funds is the canonical case. 500 only for genuinely unexpected failures.

Q105 Why 422 for insufficient funds rather than 400?

Because the request was perfectly well-formed. The JSON parsed, every field passed validation, the types were right. What failed was a **business rule** evaluated against current state - the same request might succeed tomorrow after a deposit.

400 says 'you sent something wrong, fix your request'. 422 says 'I understood you, but I cannot do it'. The distinction lets a client tell a bug from a legitimate refusal.

Q106 Describe your error contract.

One shape from every endpoint: timestamp, status, error, a machine-readable code, a human-readable message, the request path, and an optional list of field violations. The code field is the important part - it lets the React client branch on INSUFFICIENT_FUNDS or DAILY_LIMIT_EXCEEDED without parsing English, and it stays stable even if the message wording changes.

It is produced centrally by a @RestControllerAdvice, so no controller has a try/catch and no endpoint can accidentally invent its own error format.

```
{
  "timestamp": "2026-08-08T07:18:24Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "code": "INSUFFICIENT_FUNDS",
  "message": "Insufficient funds in account 900100100101. Available: 9500.00, requested: 9600.00",
  "path": "/api/v1/transactions/withdraw",
  "violations": null
}
```

Q107 How do you version the API, and why that way?

URI versioning - /api/v1/. It is the most visible option: you can see the version in a log line, a browser address bar or a curl command, and it is trivial to route at a proxy. The alternatives are a custom header or content negotiation through the Accept header, which are arguably purer but much harder to debug and to cache.

The rule I would follow is that a new version is only for breaking changes; adding an optional field is backwards-compatible and does not warrant one.

Q108 Why do you return DTOs instead of entities?

Four reasons, in order of importance. **Security** - serialising the User entity would expose password_hash and the internal numeric id. **Mass assignment** - binding a request straight to an entity lets a caller set role or balance by including them in the JSON. **Lazy loading** - serialising an entity outside a transaction throws LazyInitializationException on every lazy association. **Coupling** - without DTOs, renaming a database column becomes a breaking API change.

Q109 How does pagination work in your API?

Page and size query parameters, with size capped server-side so a client cannot request a million rows. The response is my own PageResponse envelope - content, page, size, totalElements, totalPages, first, last - rather than Spring's Page, whose JSON shape is an implementation detail that has changed between versions.

For very large datasets offset pagination degrades, because the database still walks the skipped rows; cursor-based pagination on (created_at, id) would be the next step.

Q110 How would you design a bulk transfer endpoint?

POST /transfers/batch taking a list, with one idempotency key per item and one for the batch. The key question is atomicity: all-or-nothing, or best-effort per item? For payroll, all-or-nothing in a single transaction. For a bill-payment batch, per-item, returning 207 Multi-Status with a result for each.

I would also make it asynchronous above a certain size - accept it, return 202 with a job id, and let the client poll - because holding row locks across hundreds of accounts in one transaction would block everyone else.

Q111 Why is the dashboard a single endpoint instead of several?

Because the landing screen would otherwise fire five or six parallel requests on every page load, each re-authenticating, opening its own transaction and hitting the same tables. One aggregated endpoint does the work once.

The trade-off is a less granular, more coupled contract - a purist REST design would expose the pieces separately. For a screen whose components are always rendered together, the round trips matter more than the purity. GraphQL exists largely to solve this tension properly.

Q112 How do you document the API?

springdoc-openapi generates an OpenAPI 3 document from the controllers, DTOs and validation annotations, served through Swagger UI at /swagger-ui.html. Because it is generated from the code rather than written alongside it, it cannot drift out of date - which is the failure mode of every hand-maintained API document I have seen. @Operation and @Tag annotations add the prose, and the bearer-token scheme is configured so the UI can authorise and make live calls.

CHAPTER 9

React and Frontend

Components, state, performance, and design

9

Q113 Why React, and why a single-page app rather than server-rendered pages?

React's component model suits an interface with many small, repeated pieces - account cards, transaction rows, stat tiles - and its ecosystem is what most teams actually use. An SPA fits banking specifically because the app is highly interactive and stateful: navigating between dashboard, transfer and history should not reload the page or re-fetch the shell.

The cost is a larger initial bundle and no SEO, neither of which matters for an authenticated banking app where every page is behind a login.

Q114 Explain `useState`, `useEffect` and `useMemo`, with real uses from this project.

`useState` holds state that should trigger a re-render - form fields, modal visibility, the current page number. **`useEffect`** runs side effects after render and returns a cleanup function - fetching data, subscribing, and resetting the page number to zero when a filter changes. **`useMemo`** caches an expensive computation between renders - the password-strength evaluation on the register form, which would otherwise re-run four regexes on every keystroke of any field.

The dependency array is where bugs live: omit a dependency and you capture a stale value; include an object recreated each render and the effect loops forever.

Q115 Why did you write a custom `useApi` hook instead of using React Query?

Because the app's needs are loading, error, data and a manual refetch - about forty lines. React Query is excellent and I would reach for it if I needed caching, background refetching, optimistic updates or request deduplication across components. Adding a dependency to avoid forty lines, and then only using a tenth of it, is not a good trade.

The one subtlety the hook does handle is an 'alive' ref, so a response arriving after the component unmounts does not call `setState` on a dead component.

Q116 How does your silent token refresh work, and what is the tricky part?

An axios response interceptor catches a 401, exchanges the refresh token for a new pair, and replays the original request - so a user reading a page for twenty minutes is never bounced to the login screen.

The tricky part is a shared promise. The dashboard fires several requests in parallel. If all of them 401 at once and each starts its own refresh, the server - which **rotates** refresh tokens on every use - accepts the first and rejects the rest, and the user is thrown out mid-session. Collapsing them onto a single in-flight promise is what makes rotation and parallel requests coexist. There is also a `_retried` flag so a request is never retried twice.

```
let refreshPromise = null

refreshPromise = refreshPromise || axios
  .post("/api/v1/auth/refresh", { refreshToken: tokenStore.refresh() })
  .then(res => { tokenStore.save(res.data); return res.data.accessToken })
  .finally(() => { refreshPromise = null })

const freshToken = await refreshPromise
```

Q117 How do you manage global state without Redux?

React Context, because there are only two genuinely global concerns: the authenticated user and the theme. Everything else is server data fetched per screen, which does not need a client-side store.

Redux would add a store, actions, reducers and middleware to manage two values. I would reach for it - or Zustand, which is lighter - when there is substantial client-side state shared across distant components, or when time-travel debugging would genuinely help.

Q118 What is the downside of Context, and how do you mitigate it?

Every consumer re-renders when the context value changes, and the value is a new object on every provider render unless you stop it. AuthContext wraps its value in useMemo and its callbacks in useCallback, so the object identity is stable and consumers only re-render when the user or loading flag actually changes.

For a high-frequency value I would split it into two contexts - one for the rarely-changing data, one for the volatile part - so a fast-changing value does not re-render everything.

Q119 Explain your routing and route guards.

React Router 7 with nested routes. The authenticated shell - sidebar, top bar, footer - is a parent route, and every private page renders into its Outlet, so the layout is declared once rather than imported into every page. RequireAuth wraps it, redirecting unauthenticated users to login while remembering the attempted path so they can be returned there. Admin routes add an adminOnly check.

The crucial caveat: client-side guards are **UX, not security**. Anyone can edit JavaScript in their own browser. The guard stops a customer seeing a broken admin screen; it is Spring Security rejecting the request that stops them reading admin data.

Q120 How do you prevent a double-submitted form from moving money twice?

Three layers. The submit button disables while the request is in flight, which stops the common double-click. The form carries an idempotency key minted per form session, so even if two requests do escape, the server recognises the second. And the server's unique index on that column catches a genuine race that slips past its own lookup.

Only the third is a guarantee. The first two are what make the third rarely needed.

Q121 How is your design system built?

Colours are CSS custom properties, not hard-coded Tailwind classes. Light and dark are two values of the same variable rather than two parallel sets of markup, so a component written once themes itself for free and the theme toggle is a single class on the html element. Six tokens - page, card, raised, sunken, border, and the text ramp - carry the entire theme.

On top of that sits a small component library: Button, Input, Select, Card, Modal, Badge, Skeleton, EmptyState. Every page composes those rather than restyling from scratch, which is what keeps the interface consistent.

Q122 What accessibility work did you do?

Focus-visible rings left intact, because they are a keyboard user's only cue for where they are. aria-invalid and aria-describedby on invalid inputs, so a screen reader announces the error rather than only showing it in red. Labels bound to inputs by id. aria-label on icon-only buttons. The modal traps Escape and locks background scroll. prefers-reduced-motion collapses all animation. And identity in charts is never colour-alone - there is a legend with text, direct value labels on the bars, and a plus or minus sign on every amount.

Q123 Talk me through your chart choices.

The colours were validated rather than chosen by eye: the two series hues were checked for lightness band, chroma floor, colour-blind separation under simulated protanopia and deuteranopia, and contrast against the actual chart surface, in both light and dark mode.

Cash flow is an area chart with two series on **one** shared y-axis. I never use a second y-axis - it lets two series appear to cross wherever the author likes, which makes it the most misleading chart type in common use. Spending by category is horizontal bars rather than a pie, because comparing lengths against a shared baseline is far more accurate than comparing angles - and it uses a single hue, since the axis already names each category so colour has no work to do.

Q124 How did you optimise the frontend build?

Manual chunk splitting in the Vite config puts React and Recharts in their own bundles, so a change to application code does not force users to re-download 400 KB of unchanged library. The production build is served by nginx with gzip on, immutable long-lived caching for fingerprinted assets, and no-store on index.html - because if index.html were cached, browsers would keep loading the old bundle after a deploy.

The next step would be route-level code splitting with React.lazy, so the admin screens are not downloaded by customers who can never open them.

Q125 What is the difference between controlled and uncontrolled components?

In a controlled component React state is the single source of truth: the input's value comes from state and every keystroke goes through onChange. An uncontrolled component lets the DOM hold the value and you read it with a ref.

Every form here is controlled, because they all need live validation, formatting - the phone field strips non-digits as you type - and cross-field logic such as the password-strength meter. Uncontrolled is simpler and slightly faster for a form that just needs its values on submit.

Q126 Why do React lists need keys, and why not use the array index?

Keys let React match elements between renders so it can move rather than rebuild them. With the array index as the key, deleting or reordering an item shifts every subsequent key, so React reuses the wrong DOM nodes - state ends up attached to the wrong row, which in a list of financial transactions is exactly the kind of bug you never want. Every list here uses a stable id from the server.

Q127 What is the virtual DOM and how does reconciliation work?

The virtual DOM is an in-memory tree describing what the UI should look like. On a state change React builds a new tree, diffs it against the previous one, and applies the minimal set of real DOM operations - because touching the real DOM is what is actually expensive.

The diff is heuristic rather than optimal: it assumes elements of different types produce different trees, and it uses keys to match children. Understanding those two assumptions is what explains most surprising React behaviour.

Q128 How does your debounced search work and why does it matter?

useDebounce holds a value and only updates the returned copy after a quiet period - 400 ms here. The API call depends on the debounced value, so typing 'groceries' fires one request instead of nine.

It matters for more than politeness: without it, nine in-flight requests can also return out of order, so the results for 'groc' could land after the results for 'groceries' and overwrite them with stale data.

Q129 How do you handle loading and error states?

Skeleton loaders that match the final layout, so nothing shifts when data arrives - a spinner in the middle of an empty page is a worse experience than a shape that fills in. Errors surface two ways: a toast for actions the user just took, and an inline EmptyState for a page that could not load, both showing the server's human-readable message.

Empty states are treated as a first-class case, not an afterthought: no accounts, no transactions and no payees each have their own message and a call to action.

Q130 Why is the axios base URL relative?

So the same build runs everywhere. In development Vite proxies /api to localhost:8080; in production nginx proxies /api to the backend container. In both cases the browser sees a same-origin request, so CORS never enters the picture and there is no API URL baked into the bundle that would need a rebuild to change between environments.

CHAPTER 10

Testing

What is tested, how, and why it is convincing

10

Q131 What is your testing strategy?

Three layers, matching the classic pyramid. Nine **unit tests** with mocked repositories cover the business rules - minimum balance, overdraft, frozen accounts, daily limits, ownership - and run in about two seconds. Three **concurrency integration tests** start real threads against a real database to verify the locking. Fifteen **API integration tests** drive the full HTTP stack through MockMvc, so filters, security rules, validation and the error contract all participate.

27 tests total, all passing. The distribution is deliberate: fast tests for logic, slow tests only where the interaction is the thing being tested.

Q132 What is the difference between a unit test and an integration test here?

A unit test isolates one class and mocks its collaborators, so a failure points at exactly one place and the test runs in milliseconds. TransactionServiceTest mocks the repositories and asserts that a withdrawal breaking the savings minimum balance throws.

An integration test exercises real components together. TransferConcurrencyIT needs a real database because the behaviour under test - row locking - does not exist in a mock. Mocking the thing you are trying to verify would make the test worthless.

Q133 Walk me through your concurrency test.

It creates 20 tasks, each of which authenticates its own thread - the SecurityContext is thread-local, so the main thread's authentication is invisible to workers - then waits on a CountdownLatch. Releasing the latch starts all 20 at the same instant, which maximises real contention rather than hoping for it.

Then it asserts the **invariants**: the sum of both balances equals the sum they started with, the sender dropped by exactly the total moved, and exactly 40 ledger rows exist for 20 transfers. Testing the invariant rather than the implementation means the test stays valid if I change how the locking works.

Q134 Why is that test not annotated @Transactional?

Because a transactional test wraps everything in one transaction and rolls it back at the end. The worker threads would each run in their own transaction and would never see the setup data, and their commits would be invisible to the assertions. The very interaction being tested would be eliminated by the annotation. So it commits real data and cleans up in @AfterEach.

Q135 What did your tests actually catch?

Two real bugs. The unknown-endpoint test expected 404 and got 500 - Spring 6.1 raises NoResourceFoundException for an unmatched path, not the NoHandlerFoundException I was handling, so a typo'd URL fell through to the catch-all. A 500 tells an on-call engineer the server is broken; the correct answer is 404.

And the concurrency work started from a manual load test rather than a unit test: twelve parallel transfers, nine 500s. That one led to the whole retry design.

Q136 How do you test secured endpoints?

SecurityAndApiIT uses MockMvc with the full application context, so the real filter chain runs. It logs in through /auth/login to get a genuine token and sends it as a bearer header - rather than using @WithMockUser, which would bypass the JWT filter entirely and therefore would not test the thing I care about.

That lets it assert real behaviour: that a refresh token is rejected as an access token, that a customer gets 403 on admin endpoints, and that cross-customer access returns 404 rather than 403.

Q137 What is Mockito and what do the main annotations do?

A mocking framework. @Mock creates a mock, @InjectMocks builds the class under test and injects the mocks into it, when(...).thenReturn(...) stubs behaviour, and verify() asserts an interaction happened - or, just as usefully, verify(..., never()) asserts it did not. Several tests here assert that no ledger row was written when a rule rejected the request, which is stronger than only checking the exception.

Q138 What is your test coverage, and is coverage a good metric?

The business logic and the API surface are well covered; I have not chased a percentage. Coverage measures which lines executed, not whether they were verified - a test that calls every method and asserts nothing scores 100%. It is useful as a **negative** signal: 20% coverage tells you something real. Above roughly 70% it mostly tells you people are writing tests to raise the number. I would rather have 27 tests that each assert something meaningful than 200 that inflate a metric.

Q139 Why H2 in tests rather than PostgreSQL, and what does that cost you?

H2 in memory is fast and needs no infrastructure, so the whole suite runs anywhere with no setup. The cost is real, though: H2's locking semantics differ from PostgreSQL's. Under contention H2 aborts a transaction with a serialization failure where PostgreSQL would block on the row lock - which is exactly the behaviour my concurrency tests are trying to verify.

So the tests prove the invariants hold under a **harsher** engine than production, which is not worthless, but Testcontainers running real PostgreSQL is the correct answer and is top of my roadmap.

Q140 How would you load test this?

k6 or Gatling, with a scenario weighted like real traffic - mostly dashboard and history reads, a smaller share of transfers - ramping until latency degrades. The metrics that matter are p95 and p99 latency rather than the mean, error rate by status code, and database connection pool saturation.

My specific hypothesis is that the pool would be the first limit, because a money-movement request holds two connections at its peak thanks to the REQUIRES_NEW audit write. I would want to measure that rather than assume it.

CHAPTER 11

Docker and Deployment

Containers, configuration, and operations

11

Q141 What is Docker and why containerise this?

Docker packages an application with its runtime and dependencies into an image that runs identically anywhere the Docker engine runs. For this project it means one command starts PostgreSQL, the API and the web tier, correctly wired - so a reviewer does not need Java, Node, Maven and PostgreSQL installed and version-matched to see it work.

The deeper benefit is that the environment becomes code: the Dockerfile is a reviewable, versioned description of what the runtime actually contains.

Q142 Explain your multi-stage Dockerfile.

Stage one compiles with a full JDK and Maven. Stage two starts from a JRE-only base and copies just the jar across. The build toolchain never reaches the running container, which takes the image from roughly 800 MB to about 200 MB and removes a large amount of software from the attack surface.

The layer ordering matters too: the POM is copied and dependencies resolved **before** the source is copied. Docker caches per layer, so editing a Java file does not re-download the entire dependency tree.

```
FROM maven:3.9-eclipse-temurin-21 AS build
COPY pom.xml .
RUN mvn -B dependency:go-offline          # cached unless the POM changes
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine AS runtime
RUN addgroup -S novabank && adduser -S novabank -G novabank
COPY --from=build /build/target/*.jar app.jar
USER novabank
```

Q143 Why does the container run as a non-root user?

Because by default a container process runs as root, and root inside the container maps to a genuinely privileged user in several escape scenarios. If the application is compromised, the attacker should not start out as root. Creating an unprivileged user and switching to it with USER is two lines and removes a whole class of escalation.

Q144 What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT sets the executable that always runs; CMD provides default arguments that a docker run argument can override. Using both - ENTRYPOINT for the binary and CMD for defaults - is the flexible pattern. The backend uses an ENTRYPOINT with exec so the JVM becomes PID 1 and receives SIGTERM directly, which is what allows a graceful shutdown rather than being killed after the timeout.

Q145 Explain depends_on with condition: service_healthy.

Plain depends_on only waits for the container to **start**, not to be ready. PostgreSQL takes several seconds to initialise, and the API runs Flyway migrations at boot - so it would connect to a database that is not accepting

connections yet and fail.

condition: service_healthy makes Compose wait for the container's healthcheck to pass. That removes the race properly, instead of the usual workaround of adding a sleep and hoping.

```
backend:
  depends_on:
    db:
      condition: service_healthy    # not just "started"
```

Q146 What is a Docker volume and why does the database need one?

A container's filesystem is ephemeral - remove the container and its writes are gone. A named volume is storage managed by Docker outside the container's lifecycle, so PostgreSQL's data survives docker compose down. docker compose down -v is the explicit way to discard it, which is a useful reset during development but is precisely what you must never run in production.

Q147 How do you manage configuration and secrets?

Every tunable binds to a typed AppProperties record and is overridable by an environment variable, so the same jar runs in development, Docker and production without a rebuild. That is the twelve-factor principle of strict config-code separation.

The secret in application.yml is a labelled development default. The application refuses to start if the JWT secret is under 32 characters, because HS256 needs 256 bits of key material and a short key would sign weakly rather than fail loudly. Environment variables are still not the right answer for production - they leak into process listings and crash dumps - so a real deployment would use a secrets manager.

Q148 Why does the frontend image use nginx instead of serving from Node?

Because the production output is static files, and nginx serves static files far better than Node does. The final image contains no Node, no node_modules and no source - just the built assets and a web server.

nginx also does three other jobs: it proxies /api to the backend so the browser sees everything as same-origin and CORS never applies, it applies the SPA fallback so refreshing on /dashboard serves index.html instead of 404, and it sets the security headers and cache policy.

Q149 Why does index.html have no-store while assets are cached for a year?

Vite fingerprints asset filenames - index-a1b2c3.js - so a new build produces a new name and the old one can be cached forever with no risk. index.html is the one file with a stable name, and it is what points at the fingerprinted assets. If it were cached, browsers would keep loading the previous bundle after a deploy and users would be stuck on the old version until the cache expired.

Q150 What would your CI/CD pipeline look like?

On every push: build, run the full test suite, run a dependency vulnerability scan, and build the Docker images. On merge to main: push the images to a registry tagged with the commit SHA, deploy to staging, run a smoke test, then a gated promotion to production.

For a system that moves money I would want migrations applied as an explicit step with a reviewed plan rather than silently at application boot, and I would want the deployment to be rolling with health checks so a bad build never takes the whole service down.

CHAPTER 12

Behavioural and Closing

Questions about you, answered through the project

12

Q151 Why should we hire you?

I do not stop at 'it works'. This project ran correctly before I load-tested it - no money was ever lost - but twelve concurrent transfers returned nine 500s, and I treated that as a defect rather than an edge case. Finding it meant deliberately trying to break my own system; fixing it meant understanding why a retry inside a rolled-back transaction is useless.

I also document what I build and I am honest about its limits. The documentation for this project has a chapter listing everything it does not do.

Q152 Tell me about a time you got something wrong.

I set the transfer to REPEATABLE_READ isolation, reasoning that stronger isolation meant safer money. It was cargo-culting. The explicit `SELECT ... FOR UPDATE` already prevents the balance changing underneath the transaction, so the higher isolation level added no safety - while making the database considerably more likely to abort the transaction with a serialization failure under load.

I removed it and documented why. The lesson was that 'stronger' is not automatically 'better' when you do not know what the mechanism actually does.

Q153 How do you approach learning something new?

By building the smallest thing that forces me to understand the mechanism rather than the API. For JWT I did not want to know which library method to call - I wanted to know why a self-contained token cannot be revoked, because that constraint is what drives the entire two-token design. Once you understand the constraint, the API is trivial; if you only learn the API, you will make a design mistake you cannot see.

Q154 How do you handle code review feedback?

I want the feedback that finds a bug, and I would rather be told early than ship it. Where I disagree, I ask what failure the reviewer is picturing - usually they have seen a scenario I have not, and if they have not, the conversation makes the reasoning explicit for whoever reads the code next. Either way something improves.

Q155 What do you do when you are stuck?

Reduce it until the failure is reproducible in isolation. With the concurrency 500s, the important step was writing a script that fired twelve simultaneous transfers and printed the balances before and after - that turned 'transfers sometimes fail' into 'nine of twelve return 500 with `SQLSTATE 40001`', which is a solvable problem. Then read the actual stack trace rather than guessing, and only then search.

Q156 Where do you see yourself in three years?

Owning a service end to end - design, delivery and the production consequences of both. What I enjoyed most about this project was the part where a design decision had an observable consequence I could measure: change the lock ordering, and the deadlock test hangs. I want more of that feedback loop, on systems where the stakes are real.

Q157 Do you have any questions for us?

Three I would genuinely ask. How does a change reach production here - what does the pipeline look like and how long does it take? What does on-call look like, and who owns a service after it ships? And what is the largest piece of technical debt the team would fix if it had a free month? The last one tells you more about a codebase than any architecture diagram.

Q158 What is the most interesting thing you learned building this?

That correctness and a good user experience are separate problems. My locking was correct from the start - the invariant test passed even in the version that returned 500s, because no money was ever lost. But 'the data is never corrupted' and 'the feature works' are different claims, and the gap between them was where the real engineering was. Recognising that the failure was transient, that it should be retried, and that the retry could not live where I first wanted to put it, took longer than writing the original transfer.

158 questions · NovaBank Bank Management System
Gurudutta Pradhan · guruduttapradhan140@gmail.com